

Skillab

AI Agent Development

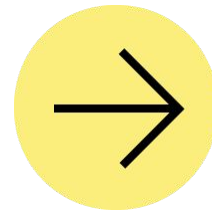
Richard Șerban Francu

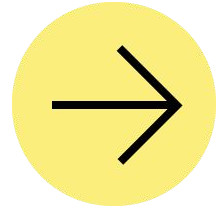
Senior AI Engineer @UCE Systems



Programa cursului

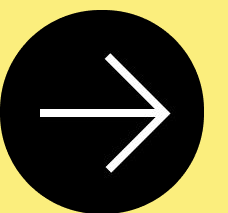
1. Noțiuni de bază — API-uri LLM și primul agent
2. Instrumente, prompturi și observabilitate
3. Procesarea și extragerea documentelor
4. Stocarea bazelor de date și RAG
5. LangGraph — Stare și fluxuri de lucru
6. Orchestrarea multi-agent
7. Optimizarea ML — Dincolo de apelurile LLM
8. Memorie și cache
9. Securitate și măsuri de protecție
- 10. Integrarea MCP**
11. Inferența SageMaker
12. Training, Fine-Tuning și RLHF





Integrarea MCP

Introducere în MCP



Ce este MCP?

Un protocol open-source de la Anthropic, lansat în noiembrie 2024

Model Context Protocol

Protocol standard pentru a conecta LLM-uri la date și capacități externe.

"USB pentru AI" — o interfață universală care funcționează cu orice client compatibil.



Open-Source

Specificație publică, oricine poate construi clients sau servere



Plug-and-Play

Un server = oricâte clients pot să-l consume fără modificări



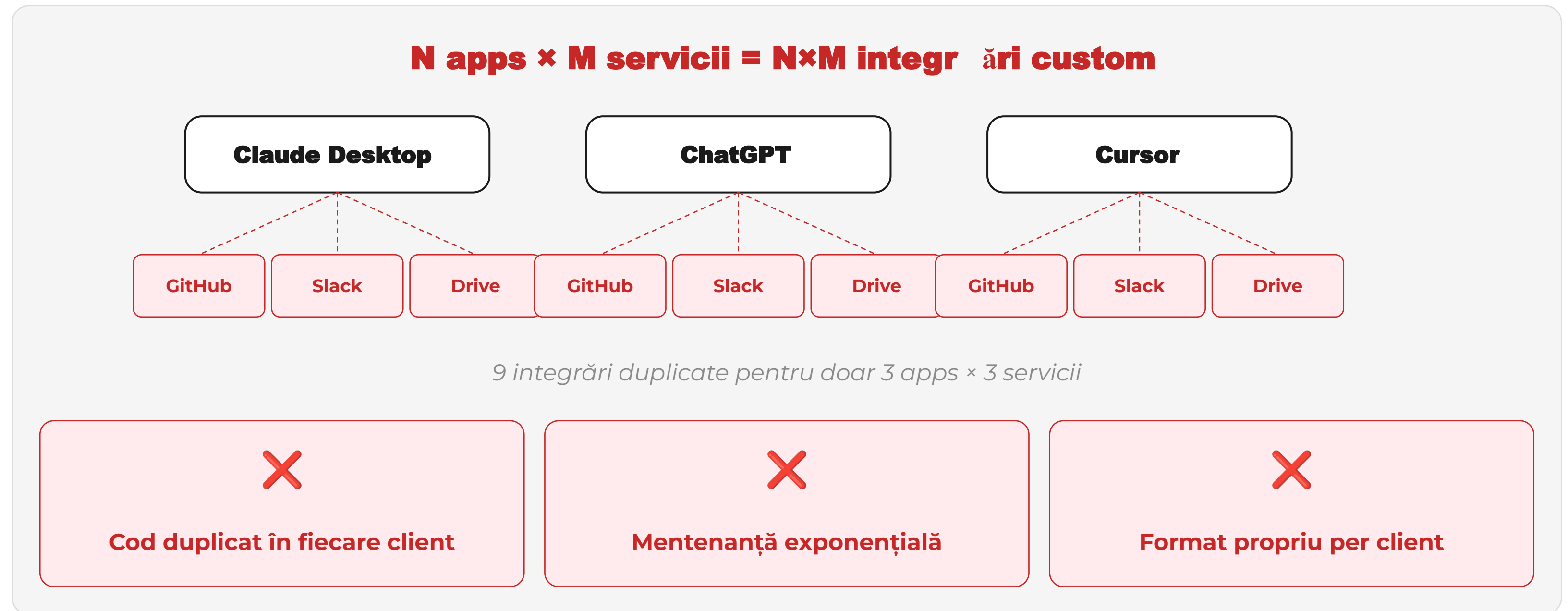
Provider-Agnostic

Nu depinde de un model anume — funcționează cu orice LLM compatibil

MCP = standard universal pentru context, tools și prompts în aplicații AI.

Problema: Context Fragmentation

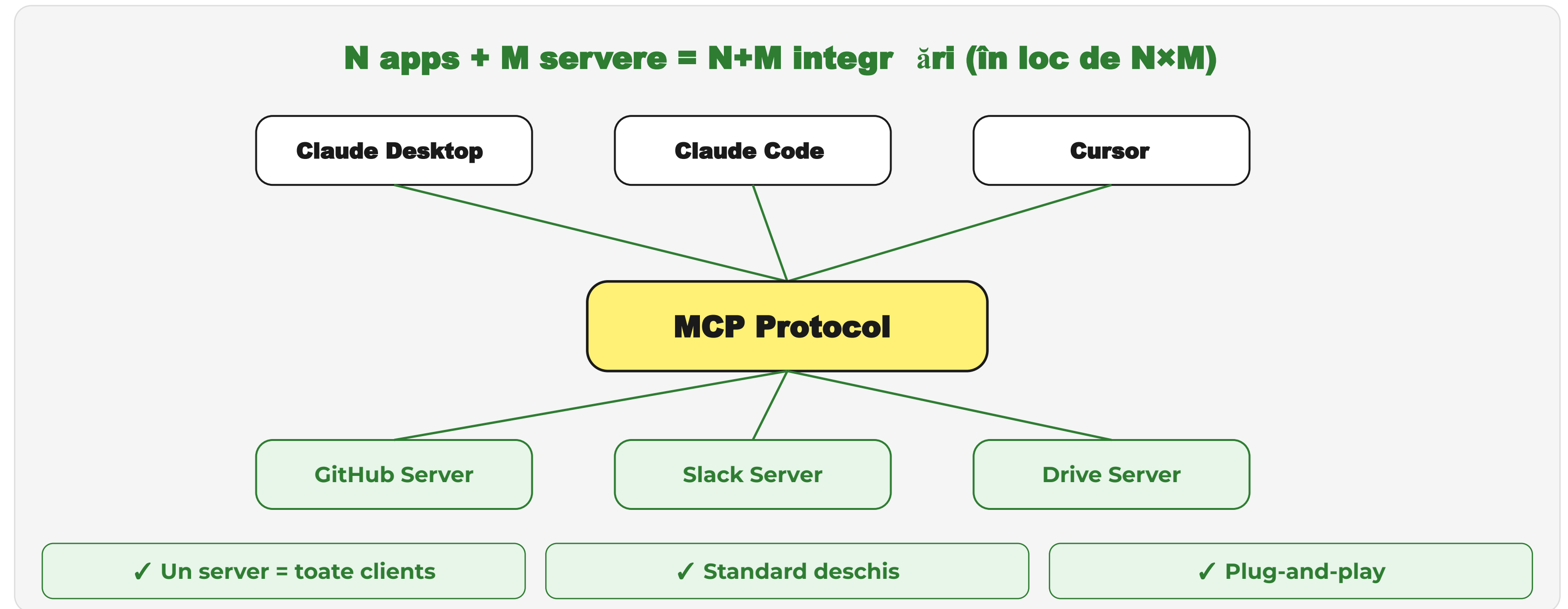
Înainte de MCP — fiecare app trebuia să-și construiască propriile integrări



Fără standard, fiecare integrare se reinventează — overhead inutil pentru toată comunitatea.

Soluția: MCP — un protocol standard

Cu MCP, fiecare server e construit O DATĂ și consumat de oricâte clients



Construiești serverul O dată — îl folosesc Claude, Cursor, Zed, oricine vorbește MCP.

Ecosistem și adopție

Cine vorbește MCP astăzi — și ce servere oficiale sunt disponibile

Clients care suportă MCP

NATIVE	Claude Desktop <i>Primul client MCP</i>
NATIVE	Claude Code <i>CLI cu MCP integrat</i>
SUPPORT	Cursor <i>IDE AI-powered</i>
SUPPORT	Continue.dev <i>Open-source AI coding</i>
SUPPORT	Zed <i>Editor modern</i>
WIP	Cody (Sourcegraph) <i>Implementare în lucru</i>

Servere oficiale

Dev Tools GitHub · GitLab · Filesystem · Puppeteer
Productivity Slack · Notion · Google Drive
Data PostgreSQL · SQLite · Memory
Web Brave Search · Fetch

Adopția e accelerată — Claude e nativ, dar ecosistemul deschis se extinde rapid.

Cele 3 componente MCP

Un server MCP poate expune oricare combinație din: Tools, Prompts, Resources

TOOLS

Acțiuni pe care LLM-ul le poate executa

LLM-ul decide când să le apeleze pe baza task-ului user-ului. Pot avea side-effects (scriu, modifică, trimit).

search_documents() · extract_data() · send_email()

PROMPTS

Template-uri reutilizabile pentru task-uri comune

User-ul le selectează direct (slash commands, meniuri). Standardizează cereri frecvente într-o organizație.

analyze_invoice · monthly_report · compare_documents

RESOURCES

Date și context pe care LLM-ul le poate citi

Read-only. Identificate prin URI. Pot fi statice (un fișier) sau dinamice (listă de docs din DB).

document://list · document://inv-001 · stats://overview

Un server poate expune toate trei — sau doar ce are sens pentru cazul tău.

Tools vs Prompts vs Resources

Cum decizi ce tip de componentă să expui — ghid rapid

Componentă	Inițiat de	Scop	Exemplu
TOOLS	Acțiuni / execuție	LLM decide	search_db(q)
PROMPTS	Template salvat	User selectează	Analizează factura
RESOURCES	Date read-only	User / LLM citește	doc://invoice.pdf

Analogie cu un API REST

```
# MCP → REST API

Tools ≈ POST/PUT/DELETE

# modifică ceva

Resources ≈ GET

# citește date

Prompts ≈ Saved Queries

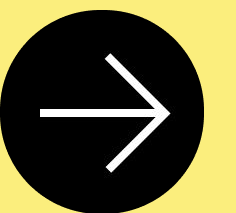
# template-uri
```

Mental model rapid când proiectezi un server.

Distincția cheie: cine inițiază interacțiunea — LLM (Tools) sau user (Prompts).

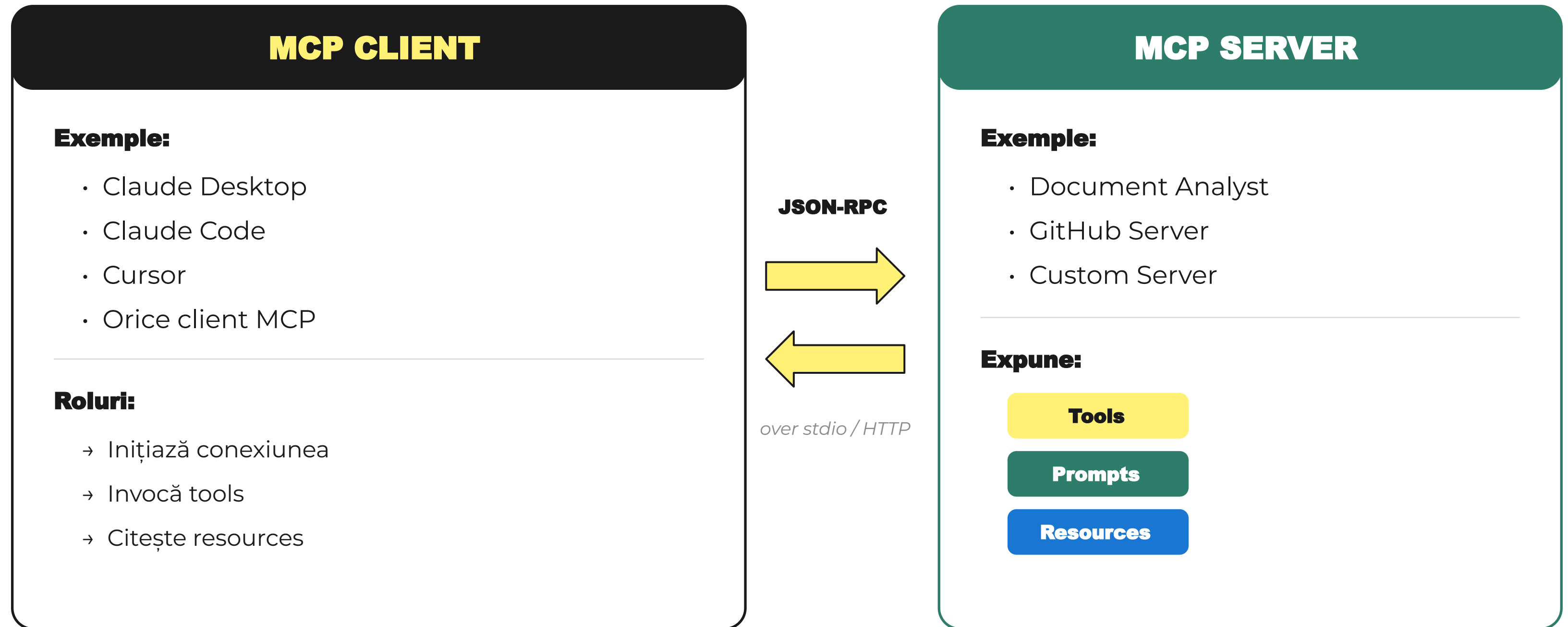
Tools = ce face LLM-ul • Prompts = ce cere user-ul • Resources = ce citește.

Arhitectura MCP



Model Client-Server

MCP folosește o arhitectură clasică client-server, cu o relație 1-la-N



1 client ↔ N servere — Claude Desktop poate avea conectate simultan GitHub, Slack, Document Analyst.

Transport Layers

MCP suportă două metode standard de comunicare client ↔ server

STDIO

Local — client pornește serverul ca subprocess, comunicare prin stdin/stdout

- ✓ Simplu, fără setup network
- ✓ Ideal pentru dezvoltare locală

- ✗ Doar local, nu remote
- ✗ Un server per client subprocess

```
$ claude-desktop → spawn → python server.py
```

HTTP+SSE

Remote — server HTTP, client face POST + primește SSE stream pentru responses

- ✓ Remote, scalabil, multi-tenant
- ✓ Firewall-friendly

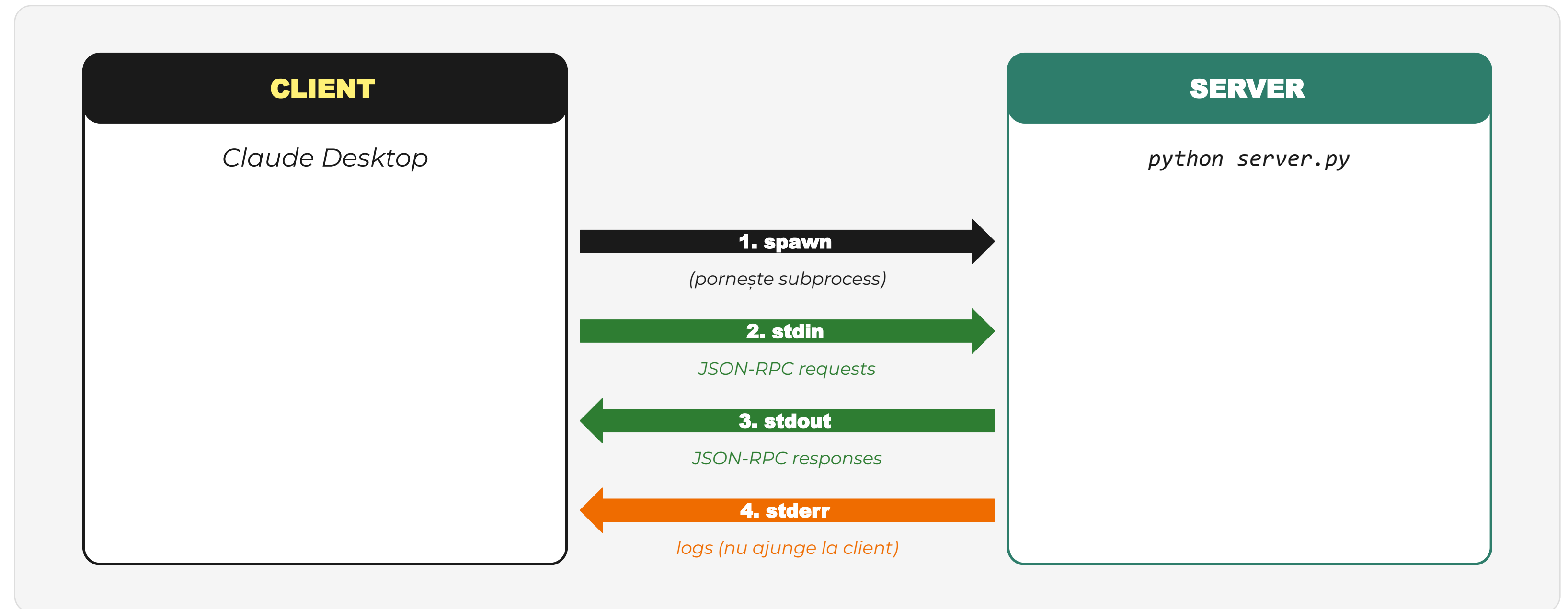
- ✗ Mai complex de pornit
- ✗ Necesită deployment + auth

```
POST https://my-mcp-server.com/sse → stream of events
```

În 90% din cazuri folosești stdio — e cel pe care îl vom folosi în lecție.

Stdio în detaliu

Cel mai comun transport pentru dezvoltare locală — flow-ul concret



⚠ **Atenție:** stdout = **DOAR JSON-RPC** . Pentru logs sau debugging — folosește `print(..., file=sys.stderr)`.

stdin = requests · stdout = responses · stderr = logs (separate, nu se amestecă).

Protocol: JSON-RPC 2.0

Toate mesajele MCP sunt JSON-RPC — 3 tipuri de bază

REQUEST

client → server

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/call",
  "params": {
    "name":
      "search_documents",
    "arguments": {
      "query":
        "factură 2024"
    }
  }
}
```

Are id → așteaptă răspuns

RESPONSE

server → client

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "content": [
      {
        "type": "text",
        "text":
          "Găsite 3 docs..."
      }
    ]
  }
}
```

Same id → corelație cu request

NOTIFICATION

fără așteptare răspuns

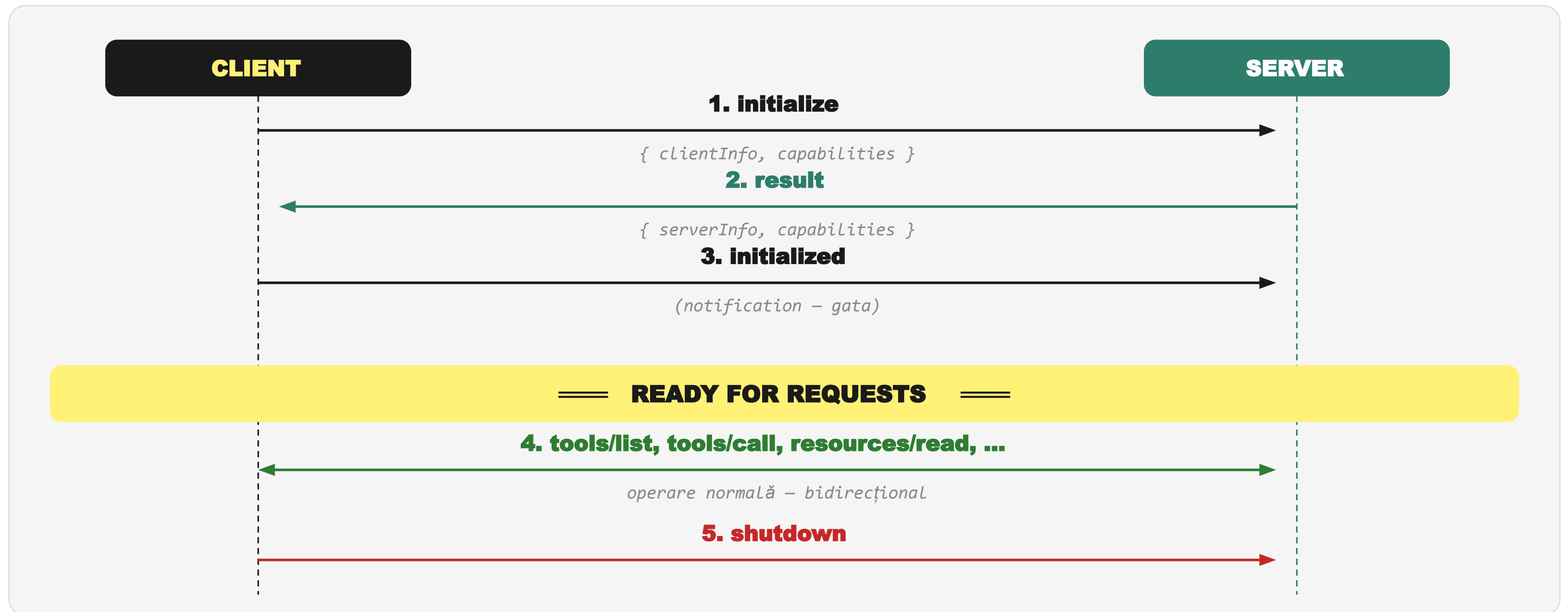
```
{
  "jsonrpc": "2.0",
  // fără id !
  "method":
    "notifications/
resources/updated",
  "params": {
    "uri":
      "document://list"
  }
}
```

Fără id → fire-and-forget

Toate mesajele = JSON-RPC 2.0 · id prezent → request · id absent → notification.

Connection Lifecycle

Etapele unei conexiuni MCP — handshake, operare, shutdown



Initialize → operate → shutdown — la fel ca orice protocol stateful.

Capabilities Negotiation

La initialize, ambele părți declară ce pot face — handshake bidirecțional

SERVER capabilities

ce poate face serverul

```
{
  "capabilities": {
    "tools": {},           // expune tools
    "prompts": {},        // expune prompts
    "resources": {
      "subscribe": true    // subscriptions
    },
    "logging": {}          // logs către client
  },
  "serverInfo": {
    "name": "document-analyst",
    "version": "1.0.0"
  }
}
```

CLIENT capabilities

ce poate face clientul

```
{
  "capabilities": {
    "roots": {
      "listChanged": true  // root dirs
    },
    "sampling": {}         // LLM sampling
  },
  "clientInfo": {
    "name": "claude-desktop",
    "version": "0.5.0"
  }
}
```

Capabilities = contract — fiecare parte știe exact ce poate cere de la cealaltă.

Metodele principale MCP

Cele 5 grupuri de metode pe care le folosești în mod normal

TOOLS

- └─ **tools/list** → listează toate tools
- └─ **tools/call** → execută un tool

PROMPTS

- └─ **prompts/list** → listează prompts
- └─ **prompts/get** → obține prompt cu args

RESOURCES

- └─ **resources/list** → listează resources
- └─ **resources/read** → citește conținut
- └─ **resources/subscribe** → abonament la changes

LIFECYCLE

- └─ **initialize** → inițializează conexiunea
- └─ **ping** → verifică serverul
- └─ **shutdown** → închide conexiunea

NOTIFICATIONS

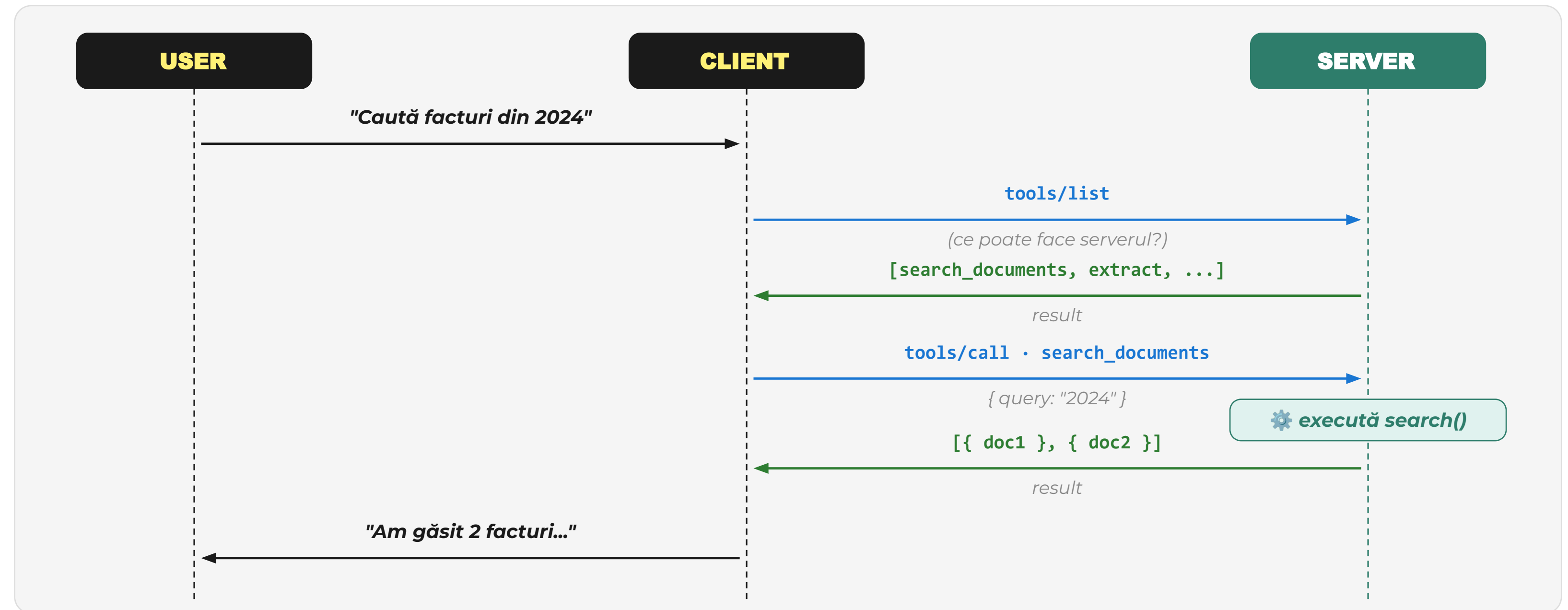
server → client (one-way)

notifications/resources/updated · **tools/list_changed** · **prompts/list_changed**

Tools/Prompts/Resources sunt request-response · Notifications sunt fire-and-forget.

Flow complet pentru un tool call

Cum se întâmplă concret când user-ul cere ceva ce trebuie executat ca tool



Tipic: 1 user message → 2 round-trips client↔server → response final.

Error handling în protocol

JSON-RPC definește coduri standard de eroare — folosește-le, nu inventa noi coduri

ERROR RESPONSE

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "error": {
    "code": -32602,
    "message": "Invalid params",
    "data": {
      "details":
        "Missing required: query"
    }
  }
}
```

ERROR CODES STANDARD

-32700	Parse error	<i>JSON invalid</i>
-32600	Invalid request	<i>Format greșit</i>
-32601	Method not found	<i>Metodă inexistentă</i>
-32602	Invalid params	<i>Args lipsă/wrong</i>
-32603	Internal error	<i>Bug în server</i>

APP-DEFINED

-32000 to **-32099** → pentru erorile tale custom

Best practice: include ``data: { details: ... }`` cu informații concrete pentru debugging.

Arhitectura în Python

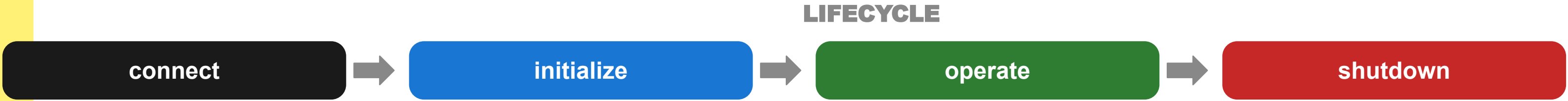
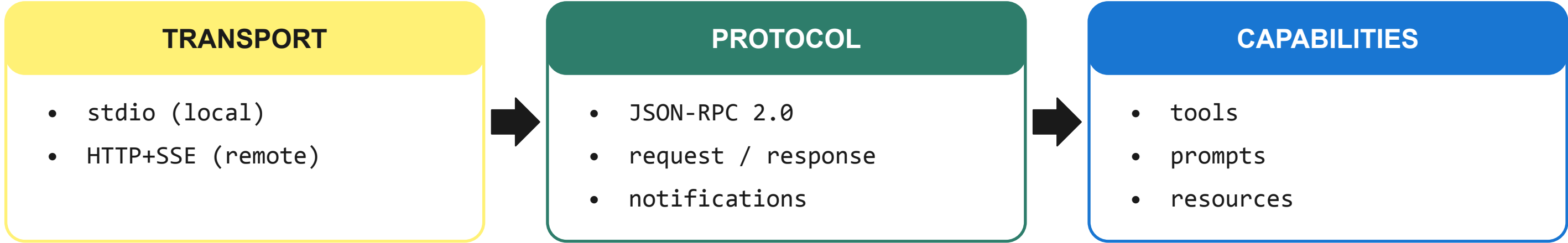
Structura minimă a unui server MCP — 3 părți: server, handlers, run loop

```
1
2 # 2. HANDLERS — declară ce expune serverul
3 @server.list_tools()
4 async def list_tools() -> list[types.Tool]:
5     return [
6         types.Tool(
7             name="search_documents",
8             description="Caută documente.",
9             inputSchema={
10                 "type": "object",
11                 "properties": {"query": {"type": "string"}},
12                 "required": ["query"],
13             },
14         )
15     ]
16
17
18 @server.call_tool()
19 async def call_tool(name: str, args: dict) -> list[types.TextContent]:
20     return [types.TextContent(type="text", text=f"ran {name}")]
21
22
23 # 3. RUN — pornește loop-ul stdio
24 async def main() -> None:
25     async with stdio_server() as (read, write):
26         await server.run(
27             read,
28             write,
29             InitializationOptions(
30                 server_name="document-analyst",
31                 server_version="1.0.0",
32                 capabilities=server.get_capabilities(
33                     notification_options=NotificationOptions(),
34                     experimental_capabilities={},
35                 ),
36             ),
37         )
38
```

3 ingrediente: Server() • @list_tools / @call_tool • stdio_server() — restul e detaliu.

Arhitectura MCP

Cele 3 layer ale protocolului + lifecycle + key takeaways



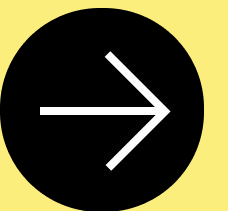
 **Protocol-agnostic** — funcționează cu orice LLM, nu doar Claude

 **Capability-based** — fiecare parte declară ce poate face

 **Bidirectional** — requests + notifications în ambele direcții

 **Stateful** — conexiunea persistă între requests

MCP Tools



Ce sunt Tools în MCP?

Tools = acțiuni pe care LLM-ul le poate invoca pentru a duce task-ul la capăt

 **USER:** "Caută toate facturile din 2024"



LLM decide să folosească tool-ul:

TOOL CALL

```
name: "search_documents"
arguments: {
  "query": "factură",
  "year": 2024
}
```



SERVER EXECUTĂ

→ query DB · filtrare · returnează rezultate structurate

 **LLM:** "Am găsit 5 facturi din 2024..."

LLM-ul vede ce tools există · alege singur când să apeleze · primește rezultate structurate.

Definirea unui tool

Fiecare tool are o schemă JSON Schema — type, properties, required, enums

```
1 import mcp.types as types
2
3 tool = types.Tool(
4     name="search_documents",
5     description="Caută documente după query și filtre",
6     inputSchema={
7         "type": "object",
8         "properties": {
9             "query": {
10                 "type": "string",
11                 "description": "Textul de căutat",
12             },
13             "doc_type": {
14                 "type": "string",
15                 "enum": ["invoice", "contract", "receipt"],
16             },
17             "limit": {"type": "integer", "default": 10},
18         },
19         "required": ["query"],
20     },
21 )
22
```

name + description + inputSchema = tot ce trebuie pentru ca LLM-ul să știe cum apelezi tool-ul.

Handler pentru tool

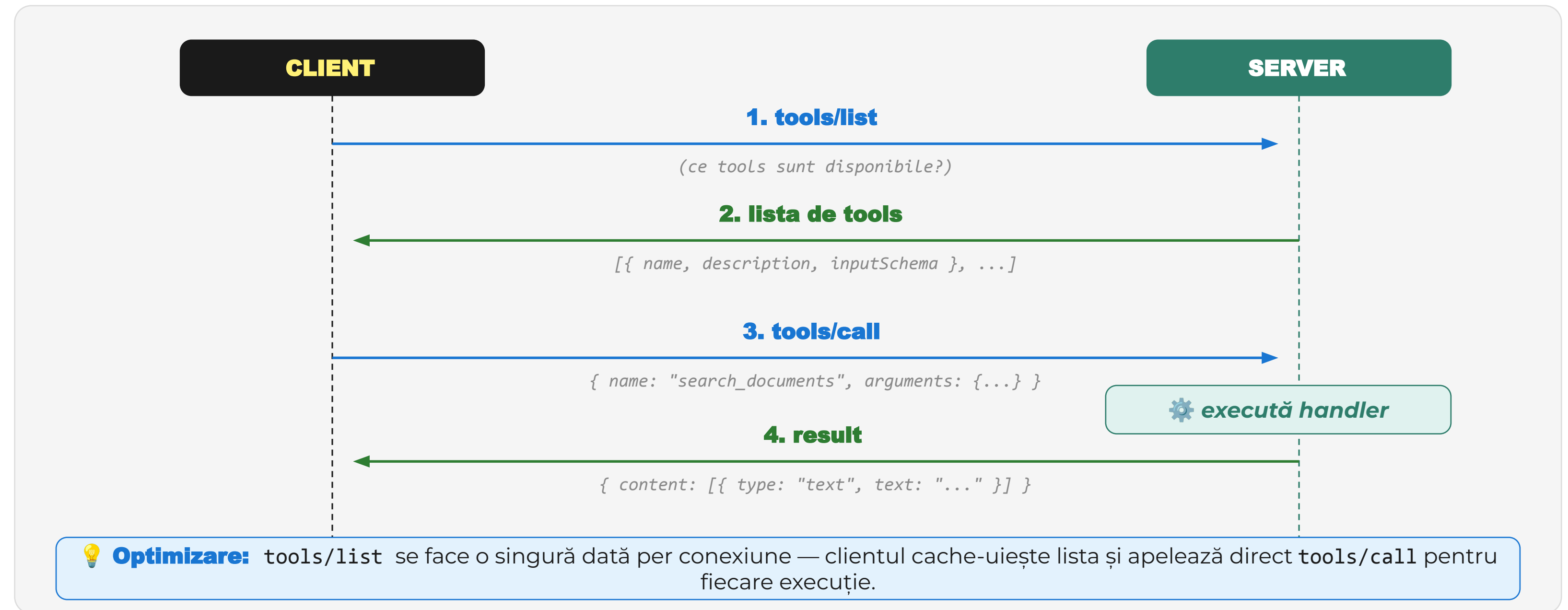
Implementarea efectivă a tool-ului — extrage args, execută logica, returnează content

```
1  @server.call_tool()
2  async def handle_tool_call(name: str, arguments: dict) -> list[types.TextContent]:
3      if name == "search_documents":
4          # Extrage parametrii — required direct, opționalii cu .get(default)
5          query = arguments["query"]
6          doc_type = arguments.get("doc_type")
7          limit = arguments.get("limit", 10)
8
9          # Execută logica (aici simulat)
10         results = await search_db(query, doc_type, limit)
11
12         # Returnează rezultat structurat ca TextContent
13         return [types.TextContent(type="text", text=f"Găsite {len(results)} documente")]
14
15         raise ValueError(f"Tool necunoscut: {name}")
16
17
18  async def search_db(query: str, doc_type: str | None, limit: int) -> list:
19      """Stub de căutare — în realitate ai interoga un index/DB."""
20      return [{"id": 1}, {"id": 2}]
21
```

Pattern: extrage args (cu defaults) → execută → returnează listă de Content.

Tool Execution Flow

Cele 5 pași concreți între client și server pentru un singur tool call



Lista se cere o dată · execuția = un singur round-trip request-response.

Tipuri de Content în răspuns

Tools pot returna 3 tipuri de content — alegi în funcție de ce produci

TextContent



Cel mai comun — răspuns text simplu, formatat sau Markdown

```
types.TextContent(  
  type="text",  
  text="Rezultatul căutării..."  
)
```

ImageContent



Pentru vizualizări — chart, screenshot, diagramă generată

```
types.ImageContent(  
  type="image",  
  data="<base64_encoded>",  
  mimeType="image/png"  
)
```

EmbeddedResource



Referință către o resursă — clientul o poate citi separat prin URI

```
types.EmbeddedResource(  
  type="resource",  
  resource=types.ResourceContents(  
    uri="document://invoice-123",  
    mimeType="application/json"))
```

Un tool poate returna o listă cu mai multe Content-uri — text + imagine + resursă, în orice combinație.

TextContent acoperă 90% din cazuri • ImageContent pentru vizualizări • EmbeddedResource pentru referințe.

Validare parametri cu JSON Schema

MCP rulează schema validation automat — nu mai scrii cod manual de validare în handler

inputSchema cu constrângeri

```
1 input_schema = {
2     "type": "object",
3     "properties": {
4         "amount": {
5             "type": "number",
6             "minimum": 0,
7             "maximum": 1_000_000,          # range numeric
8         },
9         "currency": {
10            "type": "string",
11            "enum": ["RON", "EUR", "USD"], # listă de valori permise
12        },
13        "date": {
14            "type": "string",
15            "format": "date",              # date · email · uri · uuid · regex
16        },
17    },
18    "required": ["amount", "currency"],
19 }
```

Constrângeri suportate

type

string · number · integer · boolean · object · array

enum

Listă de valori permise — ["RON", "EUR", "USD"]

minimum / maximum

Range numeric — $0 \leq \text{amount} \leq 1.000.000$

format

date · email · uri · uuid · regex pattern

Schema invalid → handler nu rulează · primești automat eroarea -32602 Invalid params.

Primul tool complet

Cele 3 ingrediente puse împreună: *list_tools* · *call_tool* · *run loop*

1

list_tools

2

call_tool

3

run

```
1 @server.list_tools()
2 async def list_tools():
3     return [types.Tool(
4         name="calculate",
5         description="Execută calcule matematice",
6         inputSchema={
7             "type": "object",
8             "properties": {"expression": {"type": "string"}},
9             "required": ["expression"],
10        },
11    )]
12
13
14 # 2. Implementează handler
15 @server.call_tool()
16 async def call_tool(name, arguments):
17     if name == "calculate":
18         result = eval(arguments["expression"]) # demo only!
19         return [types.TextContent(type="text", text=str(result))]
20     raise ValueError(f"Tool necunoscut: {name}")
21
22
23 # 3. Rulează server
24 async def main():
25     async with stdio_server() as (read, write):
26         await server.run(
27             read, write,
28             InitializationOptions(
29                 server_name="calculator-server",
30                 server_version="1.0.0",
31                 capabilities=server.get_capabilities(
32                     notification_options=NotificationOptions(),
33                     experimental_capabilities={},
34                 ),
35             ),
36         )
37
```

Cele 3 secțiuni de mai sus = șablonul universal pentru orice MCP server simplu.

Tools complexe

Un server poate expune mai multe tools — fiecare cu schema sa, toate înregistrate în `list_tools()`



document_extract

Extrage date structurate din document

required: ["file_path"]

```
inputSchema:
{ "file_path": "string",
  "extract_type":
    enum["invoice", "contract", "auto"] }
```



document_search

Caută în documentele indexate

required: ["query"]

```
inputSchema:
{ "query": "string",
  "filters": { "date_from", "date_to",
               "vendor" } }
```



document_analyze

Analizează trends în documente

required: ["metric"]

```
inputSchema:
{ "metric":
  enum["spending", "vendors", "categories"],
  "period": "string" }
```

LLM-ul vede toate 3 simultan și alege singur care tool să apeleze pentru fiecare task.

Fiecare tool = o intenție clară • LLM-ul orchestrează apelurile pentru task-uri complexe.

Best practices pentru tools

Reguli simple care fac diferența între tools pe care LLM-ul le folosește bine — sau le evită



DO

Descrieri clare și detaliate

"Caută facturi după query, vendor, interval"

Validare strictă cu JSON Schema

type, enum, format, required

Erori descriptive cu context

"Vendor 'ABC' nu există în DB"

Un tool = o acțiune clară

search · extract · analyze separate

Defaults sensibile

limit: default=10, lang: default="ro"

Returnează context util

count + summary + first 3 results



DON'T

Descrieri vagi

"Caută ceva"

Validare manuală în handler

if amount < 0 raise...

Erori generice

"Eroare"

Tools care fac prea multe

do_everything(action, params)

Prea mulți parametri required

required: [a, b, c, d, e]

Doar date brute

return [123, 456, 789]